

RnD-4: Trade-off скорость/точность в HypEx matcher для алгоритмов faiss

Контекст и постановка

Для matching-задач в HypEx нужен выбор индекса nearest-neighbor с балансом latency и recall. Требуется сравнить baseline и ANN-подходы на синтетических эмбедингах.

Вопросы и гипотезы

- H1: exact-поиск даёт лучший recall, но хуже latency на больших N.
- H2: approximate-подход (faiss при наличии) снижает latency ценой контролируемой потери recall@k.
- H3: при малых N выгоднее оставаться на exact numpy baseline.

Методология и план экспериментов

Синтетика: эмбединги базы $N \times D$ и набор запросов $Q \times D$. Метрики:

- recall@k относительно exact-эталона,
- latency на пакет запросов.

Если faiss недоступен, используется graceful fallback: сравнение exact полного поиска и approximate random-projection shortlist на numpy.

Результаты

В ноутбуке получены таблицы bench и график latency vs recall. На синтетике fallback-ANN показывает ожидаемый trade-off: ускорение при умеренном снижении recall.

Выводы и рекомендации

- Для малого каталога (до ~50k) и строгого качества: exact (numpy/faiss Flat).
- Для большого каталога: ANN с калибровкой параметров под целевой recall.
- В production держать dual-mode: быстрый ANN + периодический exact-аудит качества.

Ссылки

- docs/CONTEXT_TZ.md
- 01_bonferroni_aa_matching/report.md